



Benchmark băng thông mạng chuẩn mực

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 09

Nội dung

1. **iPerf3 là gì?** — Tại sao cần đo thay vì đoán
2. **Mô hình Client–Server** — Cách iPerf3 hoạt động
3. **Cài đặt & Khởi động server**
4. **TCP Test** — Cơ bản và nâng cao
5. **UDP Test** — Đo Jitter và Packet Loss
6. **Đọc kết quả** — Hiểu từng cột output
7. **Kịch bản thực chiến** — K8s, QoS, VPN debug
8. **Cheatsheet** — Các lệnh cốt lõi

Phần 1

iPerf3 là gì?

Vấn đề với "mạng chậm"

Khi có sự cố hiệu năng mạng, phản ứng thường gặp:

```
Developer: "Mạng chậm lắm!"  
NOC:      "Bên tôi thấy bình thường."  
Developer: "Tôi download chỉ được 50 Mbps!"  
NOC:      "Capacity là 1 Gbps, không lẽ lỗi?"
```

Vấn đề: Không ai có con số đo thực tế tại thời điểm đó.

iPerf3 giải quyết điều này: Cho phép đo throughput thực tế, jitter, packet loss giữa 2 điểm bất kỳ trong mạng — với số liệu cụ thể, không phải cảm giác.

iPerf3 đo được gì?

Metric	Mô tả	Ứng dụng
Throughput	Mbps/Gbps thực tế truyền được	Baseline capacity, bottleneck
Jitter	Dao động độ trễ (ms)	VoIP, video call, gaming
Packet Loss	% gói bị mất	Chất lượng đường truyền UDP
Retransmits	Số lần TCP gửi lại	Congestion, lỗi link
TCP Cwnd	Congestion window	TCP tuning, WAN optimization
CPU Usage	% CPU khi test	Xác định CPU bottleneck

iPerf3 vs iPerf2: Không tương thích ngược. Lệnh là `iperf3` (có số 3). Nếu gõ `iperf` — đó là phiên bản cũ.



Phần 2

Mô hình Client–Server

Cách iPerf3 hoạt động

iPerf3 bắt buộc phải có 2 đầu:



Chiều mặc định: Client → Server (upload)
--reverse / -R: Server → Client (download)
--bidir: Cả 2 chiều đồng thời (iPerf3 3.7+)

Port mặc định: 5201 TCP (control + data). Nhớ mở firewall trước khi test.

Cài đặt

```
# Ubuntu / Debian
sudo apt update && sudo apt install iperf3

# CentOS / RHEL / Rocky
sudo dnf install iperf3

# macOS (Homebrew)
brew install iperf3

# Docker (không cần cài)
docker run -it --rm networkstatic/iperf3 -s
```

Khởi động Server

```
# Server cơ bản – lắng nghe port 5201
iperf3 -s

# Server trên port tùy chỉnh
iperf3 -s -p 9999

# Daemon mode – chạy nền, cho phép nhiều kết nối liên tiếp
iperf3 -s -D

# Kiểm tra server đang chạy
ss -tlnp | grep 5201
```

Output khi server ready:

```
-----
Server listening on 5201 (test #1)
-----
```

⚠ Mở firewall: `ufw allow 5201/tcp && ufw allow 5201/udp`

Phần 3

TCP Test

TCP Test cơ bản

```
# Test 10 giây (mặc định)
iperf3 -c server_ip

# Chỉ định thời gian
iperf3 -c server_ip -t 30    # 30 giây

# Kết quả dạng JSON (dùng trong script/automation)
iperf3 -c server_ip --json | jq '.end.sum_received.bits_per_second / 1e6'
```

Output mẫu:

[ID]	Interval		Transfer	Bitrate	Retr	Cwnd
[5]	0.00-1.00	sec	112 MBytes	940 Mbits/sec	0	374 KBytes
[5]	1.00-2.00	sec	112 MBytes	940 Mbits/sec	0	374 KBytes

[5]	0.00-10.00	sec	1.09 GBytes	938 Mbits/sec	0	sender
[5]	0.00-10.00	sec	1.09 GBytes	937 Mbits/sec		receiver

TCP Test nâng cao — Parallel Streams

Tại sao cần nhiều luồng song song?

Single TCP stream bị giới hạn bởi:

Throughput tối đa = $\text{TCP Window Size} / \text{RTT}$

Ví dụ: Window 64KB / 20ms RTT = 25 Mbps ← thấp hơn capacity thực!

→ Một stream không phản ánh capacity thực của đường truyền.

→ Dùng `-P` để mở nhiều luồng song song, bypass giới hạn này.

Parallel streams – đo tổng throughput thực

`iperf3 -c server_ip -P 4` *# 4 luồng song song*

`iperf3 -c server_ip -P 8` *# 8 luồng (WAN latency cao)*

So sánh:

`iperf3 -c server -P 1` *# → 500 Mbps? (single stream bị giới hạn)*

`iperf3 -c server -P 8` *# → 950 Mbps (capacity thực)*

💡 *Rule of thumb: Dùng `-P 4` hoặc `-P 8` cho WAN test để có kết quả chính xác.*

TCP Test nâng cao — Chiều và Window

```
# Đảo chiều – đo download từ server về client
iperf3 -c server_ip --reverse      # hoặc -R
# → Đo throughput theo chiều Server → Client

# Bi-directional – cả 2 chiều cùng lúc (iPerf3 3.7+)
iperf3 -c server_ip --bidir
# → Thấy được ảnh hưởng lẫn nhau khi upload + download đồng thời

# Tăng TCP Window Size – quan trọng cho đường truyền WAN latency cao
iperf3 -c server_ip -w 4M
# Công thức:  $\text{Throughput}_{\text{max}} = \text{Window\_Size} / \text{RTT}$ 
# Window 4MB / 20ms RTT = 1.6 Gbps tối đa (không bị giới hạn)

# Giới hạn bandwidth – verify QoS / rate limiting
iperf3 -c server_ip -b 100M      # Chỉ gửi tối đa 100 Mbps

# Bỏ qua TCP Slow Start – giúp tính throughput trung bình chính xác hơn
iperf3 -c server_ip -O 3        # Không tính 3 giây đầu vào kết quả
```

Phần 4

UDP Test — Jitter & Packet Loss

Tại sao cần UDP Test?

TCP tự động:

- Retransmit gói bị mất → không thấy packet loss thực
- Điều chỉnh tốc độ → không thấy jitter

UDP không làm gì cả — gói mất là mất, trễ là trễ.

	TCP Test	UDP Test
Đo throughput	✓ Tốt	⚠ Phụ thuộc -b
Đo packet loss thực	✗ TCP che giấu	✓ Trực tiếp
Đo jitter	✗ Không có	✓ Có
Ứng dụng	File transfer, HTTP	VoIP, video, gaming

UDP Test — Lệnh và lưu ý

```
# UDP test cơ bản – PHẢI chỉ định -b
iperf3 -c server_ip -u -b 10M    # UDP, target 10 Mbps

# ⚠ QUAN TRỌNG: Không có -b → flood UDP không giới hạn
# → Gây congestion, ảnh hưởng production network
# Luôn luôn đặt -b khi test UDP!

# UDP với nhiều luồng
iperf3 -c server_ip -u -b 50M -P 4

# Test cao hơn bandwidth cho phép (độ drop rate)
iperf3 -c server_ip -u -b 200M    # Nếu link 100M → thấy loss rõ
```

Output UDP:

[ID]	Interval	Transfer	Bitrate	Jitter	Lost/Total	Datagrams
[5]	0.00–10.00s	12.5 MB	10.5 Mb/s	0.245 ms	0/8929 (0%)	

Phần 5

Đọc kết quả

Đọc TCP Output

```
[ ID] Interval          Transfer      Bitrate      Retr  Cwnd
[  5]  0.00-1.00      sec  112 MBytes  940 Mbits/sec    0   374 KBytes
[  5]  1.00-2.00      sec  112 MBytes  940 Mbits/sec    2   280 KBytes ← Retr!
-----
[  5]  0.00-10.00     sec  1.09 GBytes  938 Mbits/sec    2   sender
[  5]  0.00-10.00     sec  1.09 GBytes  937 Mbits/sec    receiver
```

Cột	Ý nghĩa	Dấu hiệu xấu
Bitrate	Throughput thực tế (Mbits/sec)	Thấp hơn kỳ vọng
Retr	Số lần TCP retransmit	> 0 = congestion hoặc packet loss
Cwnd	TCP Congestion Window	Nhỏ dần = TCP đang throttle

Retr > 0 là tín hiệu quan trọng nhất trong TCP test. Nếu Retr tăng → có gói mất → TCP giảm tốc → throughput giảm.

Đọc UDP Output

```
[ ID] Interval      Transfer  Bitrate   Jitter    Lost/Total  Datagrams
[  5] 0.00-10.00s    12.5 MB   10.5 Mb/s  0.245 ms   0/8929 (0%)  ← tốt ✓
[  5] 0.00-10.00s    12.5 MB   9.8 Mb/s   12.3 ms   89/8929 (1%) ← xấu ✗
```

Cột	Ý nghĩa	Ngưỡng
Jitter	Dao động độ trễ (ms)	VoIP: < 30ms; Gaming: < 10ms
Lost/Total	Gói mất / tổng gói	> 1% = vấn đề nghiêm trọng
Datagrams	Tổng số gói UDP đã gửi	Dùng để tính tỷ lệ mất

Cấu hình nâng cao

```
# DSCP/TOS marking – test QoS đang đánh dấu đúng không
iperf3 -c server_ip -S 0x10  # DSCP AF11 (video conferencing)
iperf3 -c server_ip -S 0x28  # DSCP EF (VoIP / low latency)

# Zero-copy mode – giảm CPU overhead khi test 10G+
iperf3 -c server_ip -Z

# IPv6
iperf3 -c server_ipv6 -6

# Lấy CPU usage trong kết quả
iperf3 -c server_ip --json | jq '.end.cpu_utilization_percent'
```

Phần 6

Kịch bản thực chiến

Scenario A: Đo throughput giữa 2 Pod trong K8s

```
# Deploy iperf3 server Pod
kubectl run iperf3-server --image=networkstatic/iperf3 -- iperf3 -s

# Lấy IP của server Pod
SERVER_IP=$(kubectl get pod iperf3-server -o jsonpath='{.status.podIP}')

# Chạy client từ Pod KHÁC NODE để test cross-node bandwidth
kubectl run iperf3-client --image=networkstatic/iperf3 --rm -it \
  -- iperf3 -c $SERVER_IP -t 30 -P 4
```

Mục tiêu: So sánh throughput giữa các CNI (Flannel vs Calico vs Cilium) hoặc kiểm tra network policy có gây overhead không.

💡 Dùng `nodeSelector` để đảm bảo server và client chạy trên 2 node khác nhau, tránh test qua localhost.

Scenario B: "Throughput thấp hơn lý thuyết dù bandwidth đủ"

```
# Chân đoán: single stream vs parallel streams
iperf3 -c server -P 1      # Kết quả: 500 Mbps?
iperf3 -c server -P 8      # Kết quả: 950 Mbps ← đây mới là capacity thực

# Nếu P1 thấp → bị giới hạn bởi TCP Window Size
# Kiểm tra: Throughput max = Window Size / RTT
# Ví dụ: Default window 64KB / RTT 20ms = 25 Mbps tối đa

# Fix: tăng window size
iperf3 -c server -w 4M -P 4
# Window 4MB / RTT 20ms = 1.6 Gbps – không còn bị giới hạn
```

Nguyên tắc: Đường WAN latency cao (> 10ms) cần window size lớn + parallel streams để đạt throughput thực sự.

Scenario C: Verify QoS / Rate Limiting

Step 1: Test baseline (không giới hạn)

`iperf3 -c server -t 10`

Kết quả: 950 Mbps

Step 2: Test với rate limit (simulate QoS policy)

`iperf3 -c server -b 100M -t 10`

Kết quả mong đợi: ~100 Mbps → rate limiting hoạt động 

Kết quả thực tế: 950 Mbps → rate limiting bị bypass hoặc sai config 

Step 3: Test DSCP marking – gói có được đánh dấu đúng không?

`iperf3 -c server -S 0x28 -t 10` # EF class (VoIP priority)

Kết hợp với tcpdump để verify DSCP field:

`tcpdump -i eth0 -v 'host server' | grep 'tos 0x'`

Scenario D: Debug VPN Throughput Thấp

```
# Baseline – ngoài VPN
iperf3 -c public_server -t 30      # → 500 Mbps

# Qua VPN
iperf3 -c vpn_server -t 30         # → 150 Mbps ← tại sao?
```

Chẩn đoán 3 nguyên nhân phổ biến:

```
# 1. CPU bottleneck (encryption overhead)
iperf3 -c vpn_server --json | jq '.end.cpu_utilization_percent'
# Nếu > 80% → CPU không đủ xử lý encryption

# 2. MTU fragmentation
ping -s 1472 -M do vpn_server      # Test MTU 1500 - 28 overhead
ping -s 1432 -M do vpn_server      # Test MTU cho WireGuard (1500 - 60)

# 3. Single stream limit (tăng parallel streams)
iperf3 -c vpn_server -P 4 -t 30   # Nếu tăng mạnh → window size issue
```

Phần 7

Cheatsheet

Bộ lệnh cốt lõi

```
# === SERVER ===
iperf3 -s                # Server cơ bản (port 5201)
iperf3 -s -p 9999        # Server port tùy chỉnh
iperf3 -s -D             # Daemon mode (nhiều kết nối)

# === TCP TEST ===
iperf3 -c server_ip      # TCP 10s, single stream
iperf3 -c server_ip -t 30 -P 4 # 30s, 4 luồng song song
iperf3 -c server_ip -P 4 -O 3 # Bỏ qua 3 giây đầu (TCP Slow Start)
iperf3 -c server_ip -R    # Đảo chiều (download test)
iperf3 -c server_ip -w 4M -P 4 # WAN test với window size lớn
iperf3 -c server_ip --bidir # Bi-directional (upload + download)

# === UDP TEST ===
iperf3 -c server_ip -u -b 10M # UDP 10 Mbps (PHẢI có -b)
iperf3 -c server_ip -u -b 50M -P 4 # UDP multi-stream

# === OUTPUT ===
iperf3 -c server_ip --json # JSON output cho automation
iperf3 -c server_ip --json | jq '.end.sum_received.bits_per_second / 1e6'
```

Quick Reference

Tình huống	Lệnh	Chú ý
Đo throughput nhanh	<code>iperf3 -c server -P 4 -t 30</code>	<code>-P 4</code> cho kết quả thực tế hơn
Download test	<code>iperf3 -c server -R -P 4</code>	Đảo chiều về client
WAN (latency cao)	<code>iperf3 -c server -w 4M -P 4</code>	Window lớn bypass giới hạn RTT
VoIP/gaming quality	<code>iperf3 -c server -u -b 10M</code>	Xem Jitter + Loss
K8s CNI benchmark	<code>-P 4 -t 30</code> giữa 2 node	Đảm bảo cross-node test
VPN debug	<code>iperf3 -c server --json</code>	Xem <code>cpu_utilization_percent</code>
QoS verify	<code>iperf3 -c server -b 100M</code>	So sánh với baseline

Đọc nhanh kết quả

TCP – Dấu hiệu xấu:

- Retr > 0 → Có packet loss / congestion
- Cwnd giảm dần → TCP đang throttle
- Bitrate << kỳ vọng → Window size nhỏ hoặc CPU bottleneck

UDP – Ngưỡng cần nhớ:

- Jitter > 30ms → VoIP bị ảnh hưởng
- Jitter > 10ms → Gaming / real-time bị lag
- Loss > 1% → Vấn đề nghiêm trọng
- Loss > 5% → Không dùng được cho VoIP

TCP Window vs RTT:

- Throughput_max = Window_Size / RTT
- Default 64KB / 20ms RTT = 25 Mbps ← dùng -w 4M để fix
- 4MB window / 20ms RTT = 1.6 Gbps ← không còn bị giới hạn

Lưu ý quan trọng

 Chi tiết	
Firewall	Mở port 5201 TCP và UDP trước khi test
UDP phải có -b	Không có <code>-b</code> → flood không giới hạn → congestion mạng
iPerf3 single-threaded	Throughput > 10 Gbps cần nhiều instance song song
Test nhiều lần	Kết quả thay đổi theo giờ cao điểm — đo ít nhất 3 lần
Production	Không chạy flood test trên mạng production
iPerf3 vs iPerf2	Lệnh <code>iperf3</code> khác <code>iperf</code> — không tương thích ngược

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

"Không đoán mò 'mạng chậm' — đo số cụ thể với `iperf3 -c server -P 4 -t 30 .`"